

程序鉴别材料

源代码（后30页）

软件名称：Cyan数据中台

版本号：V1.0

著作权人：闫晨阳

总代码行数：1505 行

```
1
2 /* ===== */
3 /* 文件: src/pages/metadata/business_db/table_schema/index.tsx */
4 /* 软件名称: Cyan数据中台 */
5 /* 版本号: V1.0 */
6 /* 著作权人: 闫晨阳 */
7 /* ===== */
8 import React, {useEffect, useState} from 'react';
9 import {
10   Button,
11   Card,
12   Col,
13   Form,
14   Input,
15   message,
16   Modal,
17   Popconfirm,
18   Row,
19   Select,
20   Space,
21   Table,
22   Tag,
23   Tooltip,
24   Typography
25 } from 'antd';
26 import {
27   DeleteOutlined,
28   EditOutlined,
29   EyeOutlined,
30   HistoryOutlined,
31   PlusOutlined,
32   TableOutlined,
33   SearchOutlined,
34   DatabaseOutlined,
35   CloudSyncOutlined,
36 } from '@ant-design/icons';
37 import PermissionButton from '@component/permission/PermissionButton';
38 import {Database, databaseApi, DatasourceType, DSApi, DsConfig, tableApi} from '@api/DSApi';
39 import {
40   listCdcConfigs,
41   createCdcConfig,
42   deleteCdcConfig,
43   toggleCdcConfig,
44   CdcConfigDTO,
45   CdcConfigCmd
46 } from '@api/CdcApi';
47 import {listSubjects, SubjectDTO} from '@api/MetadataSubjectAPI';
48 import {ColumnType} from 'antd/es/table';
49 import {useNavigate, useSearchParams} from 'react-router-dom';
50
51 const {Title, Text} = Typography;
52
53 // 环境状态枚举
54 enum EnvStatus {
55   SYNCED = 'SYNCED',
56   PENDING = 'PENDING',
```

```
57     TEST_BEHIND = 'TEST_BEHIND',
58     APPROVING = 'APPROVING',
59     REJECTED = 'REJECTED',
60 }
61
62 // 扩展的表信息 (包含前端状态)
63 interface TableInfo {
64     tableName: string;
65     tableComment: string;
66     cdcEnabled: boolean;
67     cdcConfigId?: string;
68     envStatus: EnvStatus;
69     updatedAt: string;
70 }
71
72 // 字段类型转换: MySQL/PostgreSQL 类型 -> Iceberg 类型
73 const convertToIcebergType = (dbType: string): string => {
74     const upperType = dbType.toUpperCase().split('(')[0].trim();
75
76     const commonTypeMap: Record<string, string> = {
77         'CHAR': 'STRING',
78         'VARCHAR': 'STRING',
79         'TEXT': 'STRING',
80         'MEDIUMTEXT': 'STRING',
81         'LONGTEXT': 'STRING',
82         'CLOB': 'STRING',
83         'TINYINT': 'LONG',
84         'SMALLINT': 'LONG',
85         'INT': 'LONG',
86         'INTEGER': 'LONG',
87         'MEDIUMINT': 'LONG',
88         'BIGINT': 'LONG',
89         'SERIAL': 'LONG',
90         'SMALLSERIAL': 'LONG',
91         'BIGSERIAL': 'LONG',
92         'FLOAT': 'FLOAT',
93         'REAL': 'FLOAT',
94         'DOUBLE': 'DOUBLE',
95         'DOUBLE PRECISION': 'DOUBLE',
96         'DECIMAL': 'DECIMAL',
97         'NUMERIC': 'DECIMAL',
98         'BOOLEAN': 'BOOLEAN',
99         'BOOL': 'BOOLEAN',
100        'DATE': 'DATE',
101        'TIME': 'TIME',
102        'DATETIME': 'TIMESTAMP',
103        'TIMESTAMP': 'TIMESTAMP',
104        'TIMESTAMP WITH TIME ZONE': 'TIMESTAMP_TZ',
105        'BLOB': 'BINARY',
106        'MEDIUMBLOB': 'BINARY',
107        'LONGBLOB': 'BINARY',
108        'BYTEA': 'BINARY',
109        'BINARY': 'BINARY',
110        'VARBINARY': 'BINARY',
111        'UUID': 'UUID',
112        'JSON': 'STRING',
```

```

113     'JSONB': 'STRING',
114     'ENUM': 'STRING',
115     'SET': 'STRING',
116   };
117
118   return commonTypeMap[upperType] || 'STRING';
119 };
120
121 const TableSchemaManagement: React.FC = () => {
122   const navigate = useNavigate();
123   const [searchParams] = useSearchParams();
124
125   const urlDsName = searchParams.get('dsName');
126   const urlDbName = searchParams.get('dbName');
127
128   const [datasources, setDatasources] = useState<DsConfig[]>([]);
129   const [databases, setDatabases] = useState<Database[]>([]);
130   const [tables, setTables] = useState<TableInfo[]>([]);
131
132   const [selectedDsName, setSelectedDsName] = useState<string | null>(urlDsName);
133   const [selectedDbName, setSelectedDbName] = useState<string | null>(urlDbName);
134
135   const [dsLoading, setDsLoading] = useState(false);
136   const [dbLoading, setDbLoading] = useState(false);
137   const [tableLoading, setTableLoading] = useState(false);
138
139   const [searchKeyword, setSearchKeyword] = useState('');
140
141   const [createModalVisible, setCreateModalVisible] = useState(false);
142   const [newTableName, setNewTableName] = useState('');
143
144   const [initialized, setInitialized] = useState(false);
145
146   // CDC 相关状态
147   const [cdcModalVisible, setCdcModalVisible] = useState(false);
148   const [cdcModalTableName, setCdcModalTableName] = useState<string>('');
149   const [cdcModalMode, setCdcModalMode] = useState<'create' | 'view'>('create');
150   const [cdcModalConfigName, setCdcModalConfigName] = useState<string>('');
151   const [cdcModalConfigEnabled, setCdcModalConfigEnabled] = useState<boolean>(false);
152   const [cdcForm] = Form.useForm();
153   const [subjects, setSubjects] = useState<SubjectDTO[]>([]);
154   const [cdcSubmitting, setCdcSubmitting] = useState(false);
155   const [cdcSelectedSubjectCode, setCdcSelectedSubjectCode] = useState<string>('');
156
157   useEffect(() => {
158     fetchDatasources();
159   }, []);
160
161   useEffect(() => {
162     if (datasources.length > 0 && urlDsName && !initialized) {
163       const dsExists = datasources.some(ds => ds.name === urlDsName);
164       if (dsExists) {
165         setSelectedDsName(urlDsName);
166         fetchDatabases(urlDsName);
167       }
168       setInitialized(true);

```

```

169     }
170   }, [datasources, urlDsName, initialized]);
171
172   useEffect(() => {
173     if (databases.length > 0 && urlDbName && selectedDsName === urlDsName && !selectedDbName) {
174       const dbExists = databases.some(db => db.name === urlDbName);
175       if (dbExists) {
176         setSelectedDbName(urlDbName);
177       }
178     }
179   }, [databases, urlDbName, selectedDsName, selectedDbName]);
180
181   useEffect(() => {
182     if (selectedDsName) {
183       fetchDatabases(selectedDsName);
184       if (selectedDsName !== urlDsName) {
185         setSelectedDbName(null);
186         setTables([]);
187       }
188     }
189   }, [selectedDsName]);
190
191   useEffect(() => {
192     if (selectedDsName && selectedDbName) {
193       fetchTables(selectedDsName, selectedDbName);
194     }
195   }, [selectedDsName, selectedDbName]);
196
197   const fetchDatasources = async () => {
198     setDsLoading(true);
199     try {
200       const response = await DSApi.list();
201       if (response.code === 200) {
202         setDatasources(response.data || []);
203       }
204     } catch (error) {
205       console.error('获取数据源列表失败:', error);
206       message.error('获取数据源列表失败');
207     } finally {
208       setDsLoading(false);
209     }
210   };
211
212   const fetchDatabases = async (dsName: string) => {
213     setDbLoading(true);
214     try {
215       const response = await databaseApi.list(dsName);
216       if (response.code === 200) {
217         setDatabases(response.data || []);
218       }
219     } catch (error) {
220       console.error('获取数据库列表失败:', error);
221       message.error('获取数据库列表失败');
222     } finally {
223       setDbLoading(false);

```

```

224     }
225 };
226
227 const fetchTables = async (dsName: string, dbName: string) => {
228     setTableLoading(true);
229     try {
230         const [tableRes, cdcRes] = await Promise.all([
231             tableApi.list(dsName, dbName),
232             listCdcConfigs({ dsName, dbName, syncTool: 'FLINK' })
233         ]);
234
235         let cdcMap = new Map<string, CdcConfigDTO>();
236         if (cdcRes.code === 200 && cdcRes.data) {
237             cdcMap = new Map(cdcRes.data.map(c => [`${c.dbName}.${c.tableName}`, c]));
238         }
239
240         if (tableRes.code === 200) {
241             const tableInfos: TableInfo[] = (tableRes.data || []).map(tbl => {
242                 const cdc = cdcMap.get(`${dbName}.${tbl.tableName}`);
243                 return {
244                     tableName: tbl.tableName,
245                     tableComment: tbl.tableComment,
246                     cdcEnabled: cdc?.enabled ?? false,
247                     cdcConfigId: cdc?.id,
248                     envStatus: EnvStatus.SYNCED,
249                     updatedAt: new Date().toISOString(),
250                 };
251             });
252             setTables(tableInfos);
253         }
254     } catch (error) {
255         console.error('获取表列表失败:', error);
256         message.error('获取表列表失败');
257     } finally {
258         setTableLoading(false);
259     }
260 };
261
262 const handleDsChange = (dsName: string) => {
263     setSelectedDsName(dsName);
264 };
265
266 const handleDbChange = (dbName: string) => {
267     setSelectedDbName(dbName);
268 };
269
270 // CDC 相关操作
271 const handleOpenCdcModal = async (tableName: string, configId?: string) => {
272     setCdcModalTableName(tableName);
273     cdcForm.resetFields();
274     setCdcSelectedSubjectCode('');
275
276     // 加载主题列表
277     try {
278         const subjectList = await listSubjects({parentId: '0'});
279         setSubjects(subjectList || []);

```

```

280     } catch {
281         message.error('加载主题列表失败');
282     }
283
284     if (configId) {
285         // 查看已有配置 (通过 listCdcConfigs 查询, 避免 id/name 混淆)
286         setCdcModalMode('view');
287         setCdcModalConfigName(configId);
288         try {
289             const res = await listCdcConfigs({
290                 dsName: selectedDsName!,
291                 dbName: selectedDbName!,
292                 tableName,
293                 syncTool: 'FLINK',
294             });
295             if (res.code === 200 && res.data && res.data.length > 0) {
296                 const dto = res.data[0];
297                 cdcForm.setFieldsValue({ subjectCode: dto.subjectCode });
298                 setCdcSelectedSubjectCode(dto.subjectCode || '');
299                 setCdcModalConfigEnabled(dto.enabled);
300             }
301         } catch {
302             message.error('加载 CDC 配置失败');
303         }
304     } else {
305         // 创建新配置
306         setCdcModalMode('create');
307         setCdcModalConfigName('');
308         setCdcModalConfigEnabled(false);
309         try {
310             const subjectList = await listSubjects({parentId: '0'});
311             const cdcSubject = subjectList?.find(s => s.subjectCode === 'cdc');
312             if (cdcSubject) {
313                 cdcForm.setFieldsValue({ subjectCode: cdcSubject.subjectCode });
314                 setCdcSelectedSubjectCode(cdcSubject.subjectCode);
315             }
316         } catch {
317             // ignore
318         }
319     }
320
321     setCdcModalVisible(true);
322 };
323
324 const handleCdcSubmit = async () => {
325     if (!selectedDsName || !selectedDbName) return;
326     try {
327         setCdcSubmitting(true);
328
329         if (cdcModalMode === 'view' && cdcModalConfigName) {
330             // view 模式: toggle 启用/停用
331             const res = await listCdcConfigs({
332                 dsName: selectedDsName,
333                 dbName: selectedDbName,
334                 tableName: cdcModalTableName,
335                 syncTool: 'FLINK',

```

```

336     });
337     if (res.code === 200 && res.data && res.data.length > 0) {
338         const dto = res.data[0];
339         const toggleRes = await toggleCdcConfig(dto.name, !dto.enabled);
340         if (toggleRes.code === 200) {
341             message.success(dto.enabled ? '已停用' : '已启用');
342             setCdcModalVisible(false);
343             if (selectedDsName && selectedDbName) {
344                 fetchTables(selectedDsName, selectedDbName);
345             }
346         } else {
347             message.error(toggleRes.message || '操作失败');
348         }
349     }
350 } else {
351     // create 模式: 创建新配置
352     const values = await cdcForm.validateFields();
353     const cmd: CdcConfigCmd = {
354         name: `${selectedDbName}_${cdcModalTableName}_cdc`,
355         dsName: selectedDsName,
356         dbName: selectedDbName,
357         tableName: cdcModalTableName,
358         subjectCode: values.subjectCode,
359         syncTool: 'FLINK',
360         description: `CDC 同步: ${selectedDbName}.${cdcModalTableName}`,
361     };
362
363     const res = await createCdcConfig(cmd);
364     if (res.code === 200) {
365         message.success('CDC 配置创建成功');
366         setCdcModalVisible(false);
367         if (selectedDsName && selectedDbName) {
368             fetchTables(selectedDsName, selectedDbName);
369         }
370     } else {
371         message.error(res.message || '创建失败');
372     }
373 }
374 } catch (error) {
375     console.error('CDC 操作失败:', error);
376 } finally {
377     setCdcSubmitting(false);
378 }
379 };
380
381 const handleToggleCdc = async (record: TableInfo) => {
382     if (!record.cdcConfigId) return;
383     try {
384         const res = await listCdcConfigs({ dsName: selectedDsName!, dbName: selectedDbName
385 e!, tableName: record.tableName, syncTool: 'FLINK' });
386         if (res.code === 200 && res.data && res.data.length > 0) {
387             const dto = res.data[0];
388             const toggleRes = await toggleCdcConfig(dto.name, !record.cdcEnabled);
389             if (toggleRes.code === 200) {
390                 message.success(record.cdcEnabled ? '已停用' : '已启用');
391                 if (selectedDsName && selectedDbName) {

```

```

391         fetchTables(selectedDsName, selectedDbName);
392     }
393     } else {
394         message.error(toggleRes.message || '操作失败');
395     }
396 }
397 } catch (error) {
398     console.error('CDC 启停失败:', error);
399     message.error('操作失败');
400 }
401 };
402
403 const handleCreateTable = () => {
404     setNewTableName('');
405     setCreateModalVisible(true);
406 };
407
408 const handleCreateConfirm = () => {
409     if (!newTableName.trim()) {
410         message.warning('请输入表名');
411         return;
412     }
413     navigate(`/meta/business-ds/table-schema/edit?dsName=${selectedDsName}&dbName=${selectedDbName}&tableName=${newTableName}&isNew=true`);
414     setCreateModalVisible(false);
415 };
416
417 const handleViewDetail = (tableName: string) => {
418     navigate(`/meta/business-ds/table-schema/detail?dsName=${selectedDsName}&dbName=${selectedDbName}&tableName=${tableName}`);
419 };
420
421 const handleEdit = (tableName: string) => {
422     navigate(`/meta/business-ds/table-schema/edit?dsName=${selectedDsName}&dbName=${selectedDbName}&tableName=${tableName}`);
423 };
424
425 const handleDelete = async (tableName: string) => {
426     if (!selectedDsName || !selectedDbName) return;
427
428     try {
429         const response = await tableApi.drop(selectedDsName, selectedDbName, tableName);
430         if (response.code === 200) {
431             message.success('删除成功');
432             fetchTables(selectedDsName, selectedDbName);
433         } else {
434             message.error(response.message || '删除失败');
435         }
436     } catch (error) {
437         console.error('删除表失败:', error);
438         message.error('删除失败');
439     }
440 };
441
442 const handleSyncHistory = async (tableName: string) => {
443     if (!selectedDsName || !selectedDbName) return;

```

```

444
445     try {
446         const response = await tableApi.getSchema(selectedDsName, selectedDbName, tableName);
447     if (response.code === 200 && response.data) {
448         const tableSchema = response.data;
449
450         const importData = {
451             id: '',
452             name: tableSchema.tableName,
453             owner: '',
454             subjectCode: '',
455             datasourceType: '',
456             layerCode: 'ODS',
457             comment: tableSchema.tableComment || '',
458             accessCount: '',
459             lastAccessTime: '',
460             heatLevel: '',
461             secretLevel: 'L1',
462             onlineStatus: 'ONLINE',
463             createdAt: '',
464             updatedAt: '',
465             table: {
466                 catalog: selectedDsName,
467                 schema: selectedDbName,
468                 name: tableSchema.tableName,
469                 comment: tableSchema.tableComment || '',
470                 columns: tableSchema.columns.map(col => ({
471                     name: col.name,
472                     type: convertToIcebergType(col.type),
473                     comment: col.comment || '',
474                     nullable: col.nullable ?? true,
475                     autoIncrement: col.autoIncrement ?? false,
476                     defaultValue: col.defaultValue,
477                     secretLevel: col.secretLevel || 'L1',
478                 })),
479                 indexes: tableSchema.indexes || [],
480             },
481         };
482
483         navigate('/meta/metadata/metadata_table/edit', {
484             state: {
485                 mode: 'import',
486                 importData: importData,
487             }
488         });
489     } else {
490         message.error('获取表结构失败');
491     }
492 } catch (error) {
493     console.error('获取表结构失败:', error);
494     message.error('获取表结构失败');
495 }
496 };
497
498 const getEnvStatusTag = (status: EnvStatus) => {

```

```

499     const config: Record<EnvStatus, { color: string; text: string }> = {
500         [EnvStatus.SYNCED]: {color: 'success', text: '生产已同步'},
501         [EnvStatus.PENDING]: {color: 'warning', text: '待发布'},
502         [EnvStatus.TEST_BEHIND]: {color: 'warning', text: '测试落后'},
503         [EnvStatus.APPROVING]: {color: 'processing', text: '审批中'},
504         [EnvStatus.REJECTED]: {color: 'error', text: '已拒绝'},
505     };
506     const {color, text} = config[status];
507     return <Tag color={color}>{text}</Tag>;
508 };
509
510 const getDatasourceTypeTag = (type: DatasourceType) => {
511     const colorMap: Record<DatasourceType, string> = {
512         [DatasourceType.MYSQL]: 'blue',
513         [DatasourceType.POSTGRESQL]: 'green',
514         [DatasourceType.ICEBERG]: 'purple',
515     };
516     return <Tag color={colorMap[type] || 'default'}>{type}</Tag>;
517 };
518
519 const filteredTables = tables.filter(table =>
520     table.tableName.toLowerCase().includes(searchKeyword.toLowerCase())
521 );
522
523 const columns: ColumnType<TableInfo>[] = [
524     {
525         title: '表名',
526         dataIndex: 'tableName',
527         key: 'tableName',
528         width: 140,
529         render: (name: string) => (
530             <Space>
531                 <TableOutlined style={{ color: '#8B909A' }} />
532                 <span style={{fontWeight: 500, color: '#1D2333'}}>{name}</span>
533             </Space>
534         ),
535     },
536     {
537         title: '描述',
538         dataIndex: 'tableComment',
539         key: 'tableComment',
540         width: 180,
541         ellipsis: true,
542         render: (comment: string) => (
543             <span style={{ color: '#8B909A' }}>{comment || '-'}</span>
544         ),
545     },
546     {
547         title: 'CDC状态',
548         dataIndex: 'cdcEnabled',
549         key: 'cdcEnabled',
550         width: 100,
551         render: (enabled: boolean) => (
552             <Tag color={enabled ? 'success' : 'default'}>
553                 {enabled ? '已启用' : '未启用'}
554             </Tag>

```

```

555     ),
556   },
557   {
558     title: '环境状态',
559     dataIndex: 'envStatus',
560     key: 'envStatus',
561     width: 120,
562     render: (status: EnvStatus) => getEnvStatusTag(status),
563   },
564   {
565     title: '操作',
566     key: 'action',
567     width: 160,
568     fixed: 'right',
569     render: (_, unknown, record: TableInfo) => (
570       <Space size={4}>
571         <Tooltip title="查看详情">
572           <Button
573             type="text"
574             size="small"
575             icon={<EyeOutlined />}
576             onClick={() => handleViewDetail(record.tableName)}
577             style={{ color: '#4F6DF5' }}
578           />
579         </Tooltip>
580         <Tooltip title="编辑">
581           <PermissionButton
582             type="text"
583             size="small"
584             icon={<EditOutlined />}
585             onClick={() => handleEdit(record.tableName)}
586             style={{ color: '#4E5566' }}
587             permission="MENU:meta:business-ds:table:UPDATE"
588           />
589         </Tooltip>
590         <Tooltip title="同步表结构到数仓">
591           <Button
592             type="text"
593             size="small"
594             icon={<HistoryOutlined />}
595             onClick={() => handleSyncHistory(record.tableName)}
596             style={{ color: '#1A9F5C' }}
597           />
598         </Tooltip>
599         <Tooltip title={record.cdcEnabled ? '停用 CDC' : record.cdcConfigId ? '启用
CDC' : '创建 CDC'}>
600           <Button
601             type="text"
602             size="small"
603             icon={<CloudSyncOutlined />}
604             onClick={() => {
605               if (record.cdcEnabled) {
606                 Modal.confirm({
607                   title: '确认停用 CDC 同步',
608                   content: `关闭后 ${record.tableName} 将不再同步数据到数
仓, 是否确认?`,

```

```

609         okText: '确认停用',
610         okType: 'danger',
611         cancelText: '取消',
612         onOk: () => handleToggleCdc(record),
613     });
614     } else if (record.cdcConfigId) {
615         handleOpenCdcModal(record.tableName, record.cdcConfigId);
616     } else {
617         handleOpenCdcModal(record.tableName);
618     }
619     }}
620     style={{ color: record.cdcEnabled ? '#722ED1' : '#8B909A' }}
621     />
622 </Tooltip>
623 <Popconfirm
624     title="确定删除该表吗？此操作不可恢复！"
625     onConfirm={() => handleDelete(record.tableName)}
626     okText="确定"
627     cancelText="取消"
628     >
629     <Tooltip title="删除">
630         <PermissionButton
631             type="text"
632             size="small"
633             danger
634             icon={<DeleteOutlined />}
635             permission="MENU:meta:business-ds:table:DELETE"
636         />
637     </Tooltip>
638 </Popconfirm>
639 </Space>
640     ),
641 },
642 ];
643
644 return (
645     <div className="page-container">
646         { /* 页面标题区 */ }
647         <div className="page-header-section" style={{ marginBottom: 20 }}>
648             <div className="page-title-wrapper">
649                 <Title level={4} className="page-title">表结构管理</Title>
650                 <Text className="page-subtitle">
651                     浏览和管理数据库表结构，支持表结构同步到数据仓库
652                 </Text>
653             </div>
654             <PermissionButton
655                 type="primary"
656                 icon={<PlusOutlined />}
657                 onClick={handleCreateTable}
658                 disabled={!selectedDsName || !selectedDbName}
659                 size="middle"
660                 permission="MENU:meta:business-ds:table:CREATE"
661             >
662                 新建表
663             </PermissionButton>
664         </div>

```

```

665
666      { /* 搜索筛选区 */ }
667      <Card
668          style={{
669              marginBottom: 16,
670              borderRadius: 12,
671              boxShadow: '0 2px 8px rgba(29, 35, 51, 0.03)',
672          }}
673          styles={{ body: { padding: '16px 20px' } }}
674      >
675          <Row gutter={[16, 12]} align="middle">
676              <Col>
677                  <Space>
678                      <DatabaseOutlined style={{ color: '#8B909A' }} />
679                      <Text type="secondary" style={{ fontSize: 13 }}>数据源</Text>
680                      <Select
681                          style={{ width: 200 }}
682                          placeholder="请选择数据源"
683                          value={selectedDsName}
684                          onChange={handleDsChange}
685                          loading={dsLoading}
686                          showSearch
687                          optionFilterProp="label"
688                          suffixIcon={<DatabaseOutlined style={{ color: '#8B909A', fontS
689                              size: 12 }} />}
690                      >
691                          {datasources.map(ds => (
692                              <Select.Option key={ds.name} value={ds.name} label={ds.name}
693                                  >
694                                      <Space>
695                                          {getDatasourceTypeTag(ds.datasourceType)}
696                                          {ds.name}
697                                      </Space>
698                                  </Select.Option>
699                              ))}
700                      </Select>
701                  </Space>
702              </Col>
703              <Col>
704                  <Space>
705                      <Text type="secondary" style={{ fontSize: 13 }}>数据库</Text>
706                      <Select
707                          style={{ width: 180 }}
708                          placeholder="请选择数据库"
709                          value={selectedDbName}
710                          onChange={handleDbChange}
711                          loading={dbLoading}
712                          disabled={!selectedDsName}
713                          showSearch
714                          optionFilterProp="children"
715                      >
716                          {databases.map(db => (
717                              <Select.Option key={db.name} value={db.name}
718                                  >
719                                  {db.name}
720                              </Select.Option>
721                          ))}

```

```

719             </Select>
720         </Space>
721     </Col>
722     <Col flex="auto">
723         <Input.Search
724             placeholder="搜索表名"
725             value={searchKeyword}
726             onChange={e => setSearchKeyword(e.target.value)}
727             style={{ width: 240 }}
728             allowClear
729             prefix={<SearchOutlined style={{ color: '#8B909A' }} />}
730         />
731     </Col>
732     <Col>
733         <Text type="secondary" style={{ fontSize: 13 }}>
734             共 <Text strong style={{ color: '#4F6DF5' }}>{filteredTables.length
h}</Text> 个表
735         </Text>
736     </Col>
737 </Row>
738 </Card>
739
740 {/* 数据表格 */}
741 <Card
742     style={{
743         borderRadius: 12,
744         boxShadow: '0 2px 8px rgba(29, 35, 51, 0.03)',
745         overflow: 'hidden',
746     }}
747     styles={{ body: { padding: 0 } }}
748 >
749     <Table
750         columns={columns}
751         dataSource={filteredTables}
752         rowKey="tableName"
753         loading={tableLoading}
754         pagination={{
755             pageSize: 10,
756             showSizeChanger: true,
757             showTotal: (total) => `共 ${total} 条`,
758         }}
759         locale={{emptyText: selectedDsName && selectedDbName ? '暂无数据' : '请先选
择数据源和数据库'}}
760         scroll={{ x: 700 }}
761     />
762 </Card>
763
764 {/* 新建表模态框 */}
765 <Modal
766     title="新建表"
767     open={createModalVisible}
768     onCancel={() => setCreateModalVisible(false)}
769     onOk={handleCreateConfirm}
770     width={400}
771 >
772     <Form layout="vertical" style={{ marginTop: 8 }}>

```

```

773         <Form.Item label="表名" required>
774             <Input
775                 placeholder="请输入表名"
776                 value={newTableName}
777                 onChange={e => setNewTableName(e.target.value)}
778             />
779         </Form.Item>
780     </Form>
781 </Modal>
782
783     {/* CDC 配置模态框 */}
784     <Modal
785         title={cdcModalMode === 'create' ? `创建 CDC 同步 - ${cdcModalTableName}` : `CD
C 同步 - ${cdcModalTableName}`}
786         open={cdcModalVisible}
787         onOk={handleCdcSubmit}
788         onCancel={() => setCdcModalVisible(false)}
789         confirmLoading={cdcSubmitting}
790         okText={cdcModalMode === 'view' ? (cdcModalConfigEnabled ? '停用' : '启用') :
'确定'}
791         width={520}
792     >
793         <Form
794             form={cdcForm}
795             layout="vertical"
796             style={{ marginTop: 16 }}
797         >
798             <Form.Item label="数据源">
799                 <Input value={selectedDsName || ''} disabled />
800             </Form.Item>
801             <Form.Item label="数据库">
802                 <Input value={selectedDbName || ''} disabled />
803             </Form.Item>
804             <Form.Item label="表名">
805                 <Input value={cdcModalTableName} disabled />
806             </Form.Item>
807             <Form.Item
808                 name="subjectCode"
809                 label="主题"
810                 rules={[{ required: true, message: '请选择主题' }]}
811             >
812                 <Select
813                     placeholder="请选择主题"
814                     showSearch
815                     optionFilterProp="children"
816                     onChange={(value) => setCdcSelectedSubjectCode(value as string)}
817                 >
818                     {subjects.map(subject => (
819                         <Select.Option key={subject.subjectCode} value={subject.subjectCode}
820                             {subject.subjectName} ({subject.subjectCode})
821                         </Select.Option>
822                     ))}
823                 </Select>
824             </Form.Item>
825             {cdcSelectedSubjectCode && selectedDbName && (

```

```

826         <Form.Item label="目标 ODS 表名">
827             <Input
828                 value={`ods_cdc_raw_${cdcSelectedSubjectCode.replace(/^[a-zA-Z
0-9_]/g, '_')}_${selectedDbName.replace(/^[a-zA-Z0-9_]/g, '_')}_${cdcModalTableName.replace(/^[a-zA-Z
0-9_]/g, '_')}`}
829                 disabled
830             />
831         </Form.Item>
832     )}
833 </Form>
834 </Modal>
835 </div>
836 );
837 };
838
839 export default TableSchemaManagement;
840
841
842 /* ===== */
843 /* 文件: src/pages/bi/dashboard/DashboardEditor.tsx */
844 /* 软件名称: Cyan数据中台 */
845 /* 版本号: V1.0 */
846 /* 著作权人: 闫晨阳 */
847 /* ===== */
848 import React, { useEffect, useState, useCallback, useRef } from 'react';
849 import { Button, Input, Space, message, Spin, Empty } from 'antd';
850 import { SaveOutlined, ArrowLeftOutlined, EyeOutlined, MenuFoldOutlined, MenuUnfoldOutlined }
from '@ant-design/icons';
851 import { useNavigate, useParams } from 'react-router-dom';
852 import { dashboardApi, chartApi, DashboardCmd, ChartDTO, ChartRef, ChartDataDTO, ChartType, Me
tricBiAnalysisCmd, FilterOperator, AggregateType } from '@api/DatabiApi';
853 import GridLayoutImport, { Layout } from 'react-grid-layout';
854 const GridLayout = GridLayoutImport as any;
855 import 'react-grid-layout/css/styles.css';
856 import 'react-resizable/css/styles.css';
857 import ChartLibrary from './components/ChartLibrary';
858 import PanelCard from './components/PanelCard';
859 import PanelOptions from './components/PanelOptions';
860 import FilterChart from './components/FilterChart';
861 import EChartsChart from '@pages/bi/components/EChartsChart';
862
863 const GRID_COLS = 12;
864 const ROW_HEIGHT = 50;
865 const SIDEBAR_WIDTH = 260;
866 const DEFAULT_W = 6;
867 const DEFAULT_H = 5;
868
869 const isFilterChartType = (chartType?: ChartType) => {
870     return chartType === ChartType.FILTER_SELECT || chartType === ChartType.FILTER_MULTI ||
871         chartType === ChartType.FILTER_DATE || chartType === ChartType.FILTER_DATE_RANGE;
872 };
873
874 // 找第一个不重叠的位置
875 const findFreePosition = (refs: ChartRef[], w: number, h: number): { x: number; y: number } =>
{
876     for (let y = 0; y < 100; y += h) {

```

```

877     for (let x = 0; x + w <= GRID_COLS; x += w) {
878         const overlap = refs.some(ref =>
879             ref.x < x + w && ref.x + ref.w > x &&
880             ref.y < y + h && ref.y + ref.h > y
881         );
882         if (!overlap) return { x, y };
883     }
884 }
885 return { x: 0, y: Math.max(0, ...refs.map(r => r.y + r.h), 0) };
886 };
887
888 // 检测同一行宽度是否溢出
889 const checkRowOverflow = (refs: ChartRef[]): string | null => {
890     const rows: Record<number, number> = {};
891     for (const ref of refs) {
892         for (let y = ref.y; y < ref.y + ref.h; y++) {
893             rows[y] = (rows[y] || 0) + ref.w;
894         }
895     }
896     for (const [y, total] of Object.entries(rows)) {
897         if (total > GRID_COLS) {
898             return `第 ${Number(y) + 1} 行图表总宽度(${total})超出限制(${GRID_COLS}), 请调整布局`;
899         }
900     }
901     return null;
902 };
903
904 const DashboardEditor: React.FC = () => {
905     const navigate = useNavigate();
906     const { id } = useParams<{ id?: string }>();
907     const isEdit = !!id;
908
909     const [name, setName] = useState('');
910     const [description, setDescription] = useState('');
911     const [chartRefs, setChartRefs] = useState<ChartRef[]>([]);
912     const [charts, setCharts] = useState<ChartDTO[]>([]);
913     const [chartDataMap, setChartDataMap] = useState<Record<string, ChartDataDTO>>({});
914     const [chartLoadingMap, setChartLoadingMap] = useState<Record<string, boolean>>({});
915     const [filterValuesMap, setFilterValuesMap] = useState<Record<string, string[]>>({});
916     const filterValuesMapRef = useRef<Record<string, string[]>>({});
917     const [, setLoading] = useState(false);
918     const [saving, setSaving] = useState(false);
919     const originalDslMap = useRef<Record<string, MetricBiAnalysisCmd>>({});
920
921     useEffect(() => {
922         filterValuesMapRef.current = filterValuesMap;
923     }, [filterValuesMap]);
924     const [selectedIndex, setSelectedIndex] = useState<number | null>(null);
925     const [layout, setLayout] = useState<{ i: string; x: number; y: number; w: number; h: number }[]>([]);
926     const [leftCollapsed, setLeftCollapsed] = useState(false);
927     const [rightCollapsed, setRightCollapsed] = useState(false);
928     const [canvasWidth, setCanvasWidth] = useState(960);
929
930     // 加载
931     useEffect(() => {

```

```

932     chartApi.list().then(res => { if (res.code === 200) setCharts(res.data); });
933     if (isEdit && id) {
934         setLoading(true);
935         dashboardApi.getById(id)
936             .then(res => {
937                 if (res.code === 200) {
938                     setName(res.data.name);
939                     setDescription(res.data.description || '');
940                     const refs = res.data.chartRefs || [];
941                     setChartRefs(refs);
942                     setLayout(refs.map((ref, idx) => ({ i: String(idx), x: ref.x, y: ref.
y, w: ref.w, h: ref.h })));
943                     executeAllCharts(refs);
944                 } else {
945                     message.error(res.message || '加载看板失败');
946                 }
947             })
948             .catch(() => message.error('加载看板失败'))
949             .finally(() => setLoading(false));
950     }
951 }, [isEdit, id]);
952
953 useEffect(() => {
954     const updateWidth = () => {
955         setCanvasWidth(Math.max(600, window.innerWidth - (leftCollapsed ? 0 : SIDEBAR_WIDT
H) - (rightCollapsed ? 0 : SIDEBAR_WIDTH) - 64));
956     };
957     updateWidth();
958     window.addEventListener('resize', updateWidth);
959     return () => window.removeEventListener('resize', updateWidth);
960 }, [leftCollapsed, rightCollapsed]);
961
962 // 构建合并后的 DSL (级联筛选条件)
963 const buildMergedDsl = useCallback((chartId: string): MetricBiAnalysisCmd | null => {
964     const originalDsl = originalDslMap.current[chartId];
965     if (!originalDsl) return null;
966     const ref = chartRefs.find(r => r.chartId === chartId);
967     if (!ref) return originalDsl;
968     // 收集所有影响该图表的筛选器来源:
969     // 1. 图表自身的 cascadeFrom (正向配置)
970     // 2. 哪些筛选器的 cascadeFrom 包含该图表 (反向配置)
971     const sourceIds = new Set<string>(ref.cascadeFrom || []);
972     chartRefs.forEach(filterRef => {
973         const filterChart = charts.find(c => c.id === filterRef.chartId);
974         if (filterChart && isFilterChartType(filterChart.chartType) && (filterRef.cascadeF
rom || []).includes(chartId)) {
975             sourceIds.add(filterRef.chartId);
976         }
977     });
978     const newFilters = [...(originalDsl.filters || [])];
979     for (const sourceChartId of sourceIds) {
980         const sourceChart = charts.find(c => c.id === sourceChartId);
981         if (!sourceChart || !isFilterChartType(sourceChart.chartType)) continue;
982         const values = filterValuesMapRef.current[sourceChartId] || [];
983         const dimCode = sourceChart.metricAnalysisCmd?.dimensions?.[0]?.dimCode || '';
984         if (!dimCode) continue;

```

```

985         if (values.length === 0) {
986             const idx = newFilters.findIndex(f => f.dimCode === dimCode);
987             if (idx >= 0) newFilters.splice(idx, 1);
988             continue;
989         }
990         let operator: FilterOperator;
991         if (sourceChart.chartType === ChartType.FILTER_DATE_RANGE) operator = FilterOperat
or.BETWEEN;
992         else if (values.length > 1) operator = FilterOperator.IN;
993         else operator = FilterOperator.EQ;
994         const idx = newFilters.findIndex(f => f.dimCode === dimCode);
995         if (idx >= 0) newFilters[idx] = { dimCode, operator, values };
996         else newFilters.push({ dimCode, operator, values });
997     }
998     return { ...originalDsl, filters: newFilters };
999 }, [charts, chartRefs]);
1000
1001 // 执行单个图表 (支持传入自定义 DSL)
1002 const executeChart = useCallback(async (chartId: string, dsl?: MetricBiAnalysisCmd) => {
1003     setChartLoadingMap(prev => ({ ...prev, [chartId]: true }));
1004     try {
1005         const body = dsl ? { metricAnalysisCmd: dsl } : undefined;
1006         const res = await chartApi.execute(chartId, body);
1007         if (res.code === 200 && res.data.status === 'SUCCESS') {
1008             setChartDataMap(prev => ({ ...prev, [chartId]: res.data }));
1009         }
1010     } catch { /* ignore */ }
1011     finally {
1012         setChartLoadingMap(prev => ({ ...prev, [chartId]: false }));
1013     }
1014 }, []);
1015
1016 // 执行所有图表 (无维度日期筛选框跳过 execute, 同时保存原始 DSL)
1017 const executeAllCharts = async (refs: ChartRef[]) => {
1018     const loadingMap: Record<string, boolean> = {};
1019     refs.forEach(r => loadingMap[r.chartId] = true);
1020     setChartLoadingMap(loadingMap);
1021     const dataMap: Record<string, ChartDataDTO> = {};
1022     const dslMap: Record<string, MetricBiAnalysisCmd> = {};
1023     await Promise.all(refs.map(async ref => {
1024         const chart = charts.find(c => c.id === ref.chartId);
1025         if (chart?.metricAnalysisCmd) {
1026             dslMap[ref.chartId] = JSON.parse(JSON.stringify(chart.metricAnalysisCmd));
1027         }
1028         const dimCode = chart?.metricAnalysisCmd?.dimensions?.[0]?.dimCode;
1029         const isDate = chart?.chartType === ChartType.FILTER_DATE || chart?.chartType ===
ChartType.FILTER_DATE_RANGE;
1030         if (isDate && !dimCode) return;
1031         try {
1032             const res = await chartApi.execute(ref.chartId);
1033             if (res.code === 200 && res.data.status === 'SUCCESS') dataMap[ref.chartId] =
res.data;
1034         } catch { /* ignore */ }
1035     }));
1036     originalDslMap.current = dslMap;
1037     setChartDataMap(dataMap);

```

```

1038     setChartLoadingMap({});
1039 };
1040
1041 // 筛选框值变化回调
1042 const handleFilterChange = useCallback((chartId: string, values: string[]) => {
1043     setFilterValuesMap(prev => ({ ...prev, [chartId]: values }));
1044     const sourceChart = charts.find(c => c.id === chartId);
1045     const isSourceFilter = sourceChart ? isFilterChartType(sourceChart.chartType) : false;
1046     // 查找受影响图表：
1047     // 1. 图表的 cascadeFrom 包含当前筛选器（正向配置）
1048     // 2. 当前筛选器的 cascadeFrom 包含该图表，且该图表不是筛选器（反向配置）
1049     const sourceRef = chartRefs.find(r => r.chartId === chartId);
1050     const affectedRefs = chartRefs.filter(ref => {
1051         const forward = (ref.cascadeFrom || []).includes(chartId);
1052         const targetChart = charts.find(c => c.id === ref.chartId);
1053         const reverse = isSourceFilter && targetChart && !isFilterChartType(targetChart.ch
1054 artType) && (sourceRef?.cascadeFrom || []).includes(ref.chartId);
1055         return forward || reverse;
1056     });
1057     if (affectedRefs.length === 0) return;
1058     const timer = setTimeout(() => {
1059         affectedRefs.forEach(ref => {
1060             const mergedDsl = buildMergedDsl(ref.chartId);
1061             if (mergedDsl) executeChart(ref.chartId, mergedDsl);
1062         });
1063     }, 300);
1064     return () => clearTimeout(timer);
1065 }, [charts, chartRefs, buildMergedDsl, executeChart]);
1066
1067 const handleLayoutChange = useCallback((newLayout: Layout) => {
1068     setLayout(newLayout as { i: string; x: number; y: number; w: number; h: number }[]);
1069     const nextRefs = chartRefs.map((ref, idx) => {
1070         const l = newLayout.find(item => item.i === String(idx));
1071         return l ? { ...ref, x: l.x, y: l.y, w: l.w, h: l.h } : ref;
1072     });
1073     setChartRefs(nextRefs);
1074     // 布局变化后检测溢出
1075     const overflowMsg = checkRowOverflow(nextRefs);
1076     if (overflowMsg) message.warning(overflowMsg);
1077 }, [chartRefs]);
1078
1079 const handleAddChart = (chartId: string) => {
1080     if (chartRefs.find(c => c.chartId === chartId)) {
1081         message.warning('该图表已添加');
1082         return;
1083     }
1084     const pos = findFreePosition(chartRefs, DEFAULT_W, DEFAULT_H);
1085     const newRef: ChartRef = { chartId, x: pos.x, y: pos.y, w: DEFAULT_W, h: DEFAULT_H, ti
1086 tleVisible: true, borderStyle: 'default', cascadeFrom: [] };
1087     const nextRefs = [...chartRefs, newRef];
1088     setChartRefs(nextRefs);
1089     setLayout(nextRefs.map((ref, idx) => ({ i: String(idx), x: ref.x, y: ref.y, w: ref.w,
1090 h: ref.h })));
1091     setSelectedIndex(nextRefs.length - 1);
1092     // 执行新图表（无维度日期筛选框跳过，同时保存原始 DSL）
1093     const chart = charts.find(c => c.id === chartId);

```

```

1091     if (chart?.metricAnalysisCmd) {
1092         originalDslMap.current[chartId] = JSON.parse(JSON.stringify(chart.metricAnalysisCmd));
1093     }
1094     const dimCode = chart?.metricAnalysisCmd?.dimensions?.[0]?.dimCode;
1095     const isDate = chart?.chartType === ChartType.FILTER_DATE || chart?.chartType === ChartType.FILTER_DATE_RANGE;
1096     if (!(isDate && !dimCode)) {
1097         setChartLoadingMap(p => ({ ...p, [chartId]: true }));
1098         chartApi.execute(chartId).then(res => {
1099             if (res.code === 200 && res.data.status === 'SUCCESS') setChartDataMap(p => ({
1100                 ...p, [chartId]: res.data }));
1101             setChartLoadingMap(p => ({ ...p, [chartId]: false }));
1102         });
1103     };
1104
1105     const handleDuplicateChart = (index: number) => {
1106         const ref = chartRefs[index];
1107         const pos = findFreePosition(chartRefs, ref.w, ref.h);
1108         const newRef: ChartRef = { ...ref, x: pos.x, y: pos.y };
1109         const nextRefs = [...chartRefs, newRef];
1110         setChartRefs(nextRefs);
1111         setLayout(nextRefs.map((r, idx) => ({ i: String(idx), x: r.x, y: r.y, w: r.w, h: r.h })));
1112         setSelectedIndex(nextRefs.length - 1);
1113     };
1114
1115     const handleRemoveChart = (index: number) => {
1116         const next = [...chartRefs];
1117         const removedId = next[index]?.chartId;
1118         next.splice(index, 1);
1119         setChartRefs(next);
1120         setLayout(next.map((ref, idx) => ({ i: String(idx), x: ref.x, y: ref.y, w: ref.w, h: ref.h })));
1121         if (removedId) setChartDataMap(p => { const m = { ...p }; delete m[removedId]; return m; });
1122         if (selectedIndex === index) setSelectedIndex(null);
1123         else if (selectedIndex !== null && selectedIndex > index) setSelectedIndex(selectedIndex - 1);
1124     };
1125
1126     const handleUpdateRef = (index: number, key: keyof ChartRef, value: unknown) => {
1127         const next = [...chartRefs];
1128         next[index] = { ...next[index], [key]: value };
1129         setChartRefs(next);
1130         if (key === 'w' || key === 'h') {
1131             setLayout(next.map((ref, idx) => ({ i: String(idx), x: ref.x, y: ref.y, w: ref.w, h: ref.h })));
1132         }
1133     };
1134
1135     const handleSave = async (andPreview = false) => {
1136         if (!name.trim()) { message.warning('请输入看板名称'); return null; }
1137         const overflowMsg = checkRowOverflow(chartRefs);
1138         if (overflowMsg) { message.warning(`布局有误: ${overflowMsg}, 请修正后再保存`); return null; }

```

```

1; }
1139     const cmd: DashboardCmd = {
1140         name: name.trim(), description: description || undefined,
1141         chartRefs: chartRefs.length ? chartRefs : undefined,
1142         layoutConfig: JSON.stringify({ cols: GRID_COLS, rowHeight: ROW_HEIGHT, compact: true, version: 2 }),
1143     };
1144     setSaving(true);
1145     try {
1146         let res;
1147         if (isEdit && id) res = await dashboardApi.update(id, cmd);
1148         else res = await dashboardApi.create(cmd);
1149         if (res.code === 200) {
1150             message.success(isEdit ? '更新成功' : '创建成功');
1151             const dashboardId = isEdit && id ? id : String(res.data);
1152             if (andPreview) navigate(`/bi/dashboard/view/${dashboardId}`);
1153             else if (!isEdit) navigate(`/bi/dashboard/edit/${dashboardId}`);
1154             return dashboardId;
1155         } else { message.error(res.message || '保存失败'); return null; }
1156     } catch { message.error('保存失败'); return null; }
1157     finally { setSaving(false); }
1158 };
1159
1160 const handlePreview = async () => {
1161     if (isEdit && id) navigate(`/bi/dashboard/view/${id}`);
1162     else { const did = await handleSave(true); if (!did) message.warning('请先保存看板'); }
1163 };
1164
1165 const existingChartIds = chartRefs.map(r => r.chartId);
1166
1167 // 渲染面板内容
1168 const renderContent = (chartId: string, chart?: ChartDTO) => {
1169     const isLoading = chartLoadingMap[chartId];
1170     const data = chartDataMap[chartId];
1171     if (isLoading) return <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'center', height: '100%' }}><Spin size="small" /></div>;
1172     if (!data || data.status !== 'SUCCESS') return <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'center', height: '100%' }}><Empty description="无预览数据" imageStyle={{ height: 40 }} /></div>;
1173     if (!chart) return <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'center', height: '100%', color: '#d9d9d9' }}>未知图表</div>;
1174     if (isFilterChartType(chart.chartType)) {
1175         return (
1176             <div style={{ padding: '8px 12px', height: '100%' }}>
1177                 <FilterChart chart={chart} data={data} onChange={(vals) => handleFilterChange(chart.id, vals)} />
1178             </div>
1179         );
1180     }
1181     if (chart.chartType === ChartType.NUMBER && chart.metricAnalysisCmd?.metrics?.length) {
1182         return <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'center', height: '100%' }}><div style={{ textAlign: 'center' }}><div style={{ fontSize: 28, fontWeight: 600, color: '#4F6DF5' }}>{Number(data.rows[0]?.[chart.metricAnalysisCmd.metrics[0].metricCode] ?? 0).toLocaleString()}</div><div style={{ fontSize: 12, color: '#999' }}>{chart.metricAnalysisCmd.metrics[0].metricName || chart.metricAnalysisCmd.metrics[0].alias || chart.metricAnalysisCmd.metrics[0].met

```

```

ricCode}</div></div></div>;
1183     }
1184     if (chart.chartType === ChartType.TABLE) {
1185         const aliasMap: Record<string, string> = {};
1186         chart.metricAnalysisCmd?.dimensions?.forEach(d => { aliasMap[d.dimCode] = d.dimName || d.alias || d.dimCode; });
1187         chart.metricAnalysisCmd?.metrics?.forEach(m => { aliasMap[m.metricCode] = m.metricName || m.alias || m.metricCode; });
1188         return <div style={{ height: '100%', overflow: 'auto' }}><table style={{ width: '100%', fontSize: 11, borderCollapse: 'collapse' }}><thead><tr>{data.columns.map(col => <th key={col} style={{ borderBottom: '1px solid #f0f0f0', padding: '3px 6px', textAlign: 'left', fontWeight: 500, background: '#fafafa', fontSize: 10 }}>{aliasMap[col] || col}</th>)}</tr></thead><tbody>{data.rows.slice(0, 20).map((row, idx) => <tr key={idx}>{data.columns.map(col => <td key={col} style={{ borderBottom: '1px solid #f0f0f0', padding: '3px 6px' }}>{String(row[col] ?? '')}</td>)}</tr>)}</tbody></table></div>;
1189     }
1190     if (data.rows.length > 0) {
1191         return <div style={{ width: '100%', height: '100%' }}><EChartsChart chartType={chart.chartType} columns={data.columns} rows={data.rows} dimensions={chart.metricAnalysisCmd?.dimensions?.map(d => ({ field: d.dimCode, alias: d.dimName || d.alias || d.dimCode })) || []} metrics={chart.metricAnalysisCmd?.metrics?.map(m => ({ field: m.metricCode, aggregate: AggregateType.SUM, alias: m.metricName || m.alias || m.metricCode })) || []} /></div>;
1192     }
1193     return <Empty description="暂无数据" imageStyle={{ height: 40 }} />;
1194 };
1195
1196 return (
1197     <div style={{ display: 'flex', flexDirection: 'column', height: '100%', background: '#f0f2f5' }}>
1198         { /* 顶部工具栏 */ }
1199         <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', padding: '8px 16px', borderBottom: '1px solid #e8e8e8', background: '#fff', flexShrink: 0 }}>
1200             <Space>
1201                 <Button icon={<ArrowLeftOutlined />} size="small" onClick={() => navigate('/bi/dashboard')}>返回</Button>
1202                 <Input placeholder="看板名称" value={name} onChange={e => setName(e.target.value)} style={{ width: 200 }} variant="borderless" />
1203                 <Input placeholder="描述 (可选)" value={description} onChange={e => setDescription(e.target.value)} style={{ width: 240 }} variant="borderless" />
1204             </Space>
1205             <Space>
1206                 <Button icon={<EyeOutlined />} size="small" onClick={handlePreview}>预览</Button>
1207                 <Button type="primary" icon={<SaveOutlined />} size="small" loading={saving} onClick={() => handleSave()}>保存</Button>
1208             </Space>
1209         </div>
1210
1211         { /* 主体 */ }
1212         <div style={{ display: 'flex', flex: 1, overflow: 'hidden' }}>
1213             { /* 左侧 */ }
1214             <div style={{ width: leftCollapsed ? 40 : SIDEBAR_WIDTH, flexShrink: 0, borderRight: '1px solid #e8e8e8', background: '#fff', overflow: 'hidden', display: 'flex', flexDirection: 'column', transition: 'width 0.2s' }}>
1215                 {leftCollapsed ? (
1216                     <div style={{ display: 'flex', justifyContent: 'center', padding: '12px'

```

```

x 0', cursor: 'pointer' }} onClick={() => setLeftCollapsed(false)}><MenuUnfoldOutlined style={{ color: '#666' }} /></div>
1217         ) : (
1218             <>
1219                 <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', padding: '10px 12px', borderBottom: '1px solid #f0f0f0' }}>
1220                     <span style={{ fontSize: 13, fontWeight: 500 }}>组件库</span>
1221                     <MenuFoldOutlined style={{ color: '#999', cursor: 'pointer' }}
onClick={() => setLeftCollapsed(true)} />
1222                 </div>
1223                 <div style={{ flex: 1, overflow: 'hidden' }}>
1224                     <ChartLibrary charts={charts} onAddChart={handleAddChart} existingChartIds={existingChartIds} onRefreshCharts={() => chartApi.list().then(res => { if (res.code == 200) setCharts(res.data); })} />
1225                 </div>
1226             </>
1227         )}
1228     </div>
1229
1230     { /* 画布 */ }
1231     <div style={{ flex: 1, overflow: 'auto', padding: 16, background: '#f0f2f5' }}>
>
1232         <div style={{ background: '#fff', minHeight: 600, border: '1px dashed #d9d9d9', position: 'relative', padding: 8 }}>
1233             {chartRefs.length === 0 ? (
1234                 <div style={{ display: 'flex', alignItems: 'center', justify-content: 'center', height: 400, color: '#999', fontSize: 14, border: '2px dashed #e8e8e8', borderRadius: 8, margin: 16 }}>请从左侧组件库添加图表到画布</div>
1235             ) : (
1236                 <GridLayout className="layout" layout={layout as any} cols={GRID_COLS} rowHeight={ROW_HEIGHT} width={canvasWidth} onLayoutChange={handleLayoutChange} margin={[12, 12]} isBounded={false}>
1237                     {chartRefs.map((ref, idx) => {
1238                         const chart = charts.find(c => c.id === ref.chartId);
1239                         const isSelected = selectedIndex === idx;
1240                         return (
1241                             <div key={String(idx)}>
1242                                 <PanelCard title={chart?.name || ref.chartId} selected={isSelected} bgColor={ref.bgColor || undefined} borderStyle={ref.borderStyle} titleVisible={ref.titleVisible} onClick={() => setSelectedIndex(idx)} onDuplicate={() => handleDuplicateChart(idx)} onDelete={() => handleRemoveChart(idx)} dragHandleProps={{ className: 'drag-handle' }}>
1243                                     {renderContent(ref.chartId, chart)}
1244                                 </PanelCard>
1245                             </div>
1246                         );
1247                     })}
1248                 </GridLayout>
1249             )}
1250         </div>
1251     </div>
1252
1253     { /* 右侧 */ }
1254     <div style={{ width: rightCollapsed ? 40 : SIDEBAR_WIDTH, flexShrink: 0, borderLeft: '1px solid #e8e8e8', background: '#fff', overflow: 'hidden', display: 'flex', flex-direction: 'column', transition: 'width 0.2s' }}>
1255         {rightCollapsed ? (

```

```

1256             <div style={{ display: 'flex', justifyContent: 'center', padding: '12p
x 0', cursor: 'pointer' }} onClick={() => setRightCollapsed(false)}><MenuUnfoldOutlined style={{ col
or: '#666' }} /></div>
1257             ) : (
1258                 <>
1259                     <div style={{ display: 'flex', justifyContent: 'space-between', al
ignKeyItems: 'center', padding: '10px 12px', borderBottom: '1px solid #f0f0f0' }}>
1260                         <span style={{ fontSize: 13, fontWeight: 500 }}>属性设置</span>
1261                         <MenuFoldOutlined style={{ color: '#999', cursor: 'pointer' }}
onClick={() => setRightCollapsed(true)} />
1262                     </div>
1263                     <div style={{ flex: 1, overflow: 'auto' }}>
1264                         {selectedIndex !== null && chartRefs[selectedIndex] ? (
1265                             <PanelOptions chartRef={chartRefs[selectedIndex]} chart={c
harts.find(c => c.id === chartRefs[selectedIndex].chartId)} allChartRefs={chartRefs} allCharts={char
ts} onChange={(key, value) => handleUpdateRef(selectedIndex, key as keyof ChartRef, value)} />
1266                         ) : (
1267                             <div style={{ color: '#999', textAlign: 'center', paddingT
op: 60, fontSize: 13 }}>点击画布中的面板<br />进行配置</div>
1268                         )}
1269                     </div>
1270                 </>
1271             )}
1272         </div>
1273     </div>
1274 </div>
1275 );
1276 };
1277
1278 export default DashboardEditor;
1279
1280
1281 /* ===== */
1282 /* 文件: src/pages/auth/role/index.tsx */
1283 /* 软件名称: Cyan数据中台 */
1284 /* 版本号: V1.0 */
1285 /* 著作权人: 闫晨阳 */
1286 /* ===== */
1287 import React, { useEffect, useState } from 'react';
1288 import {
1289     Table,
1290     Button,
1291     Drawer,
1292     Form,
1293     Input,
1294     Space,
1295     Tag,
1296     message,
1297     Spin,
1298     Empty,
1299     Modal,
1300     Transfer,
1301     TreeSelect,
1302     Typography,
1303     Select,
1304 } from 'antd';

```

```

1305 import {
1306     PlusOutlined,
1307     EditOutlined,
1308     DeleteOutlined,
1309     TeamOutlined,
1310 } from '@ant-design/icons';
1311 import PermissionButton from '@component/permission/PermissionButton';
1312 import {
1313     listRoles,
1314     saveRole,
1315     deleteRole,
1316     listRoleMembers,
1317     addRoleMembers,
1318     removeRoleMember,
1319     RoleDTO,
1320     RoleCmd,
1321     RoleMemberDTO,
1322     getUserFunctionPermissions,
1323     FunctionPermissionNode,
1324 } from '@api/DataAuthApi';
1325 import { listEmployees } from '@api/EmployeeApi';
1326
1327 const { Title } = Typography;
1328
1329 interface TreeDataNode {
1330     title: string;
1331     value: string;
1332     children?: TreeDataNode[];
1333 }
1334
1335 const convertPermissionTree = (nodes: FunctionPermissionNode[]): TreeDataNode[] => {
1336     return nodes.map(node => ({
1337         title: node.name,
1338         value: node.key,
1339         children: node.children ? convertPermissionTree(node.children) : undefined,
1340     }));
1341 };
1342
1343 const RolePage: React.FC = () => {
1344     const [roles, setRoles] = useState<RoleDTO[]>([]);
1345     const [loading, setLoading] = useState(false);
1346     const [drawerVisible, setDrawerVisible] = useState(false);
1347     const [editingRole, setEditingRole] = useState<RoleDTO | null>(null);
1348     const [memberModalVisible, setMemberModalVisible] = useState(false);
1349     const [currentRoleId, setCurrentRoleId] = useState<string>('');
1350     const [members, setMembers] = useState<RoleMemberDTO[]>([]);
1351     const [memberLoading, setMemberLoading] = useState(false);
1352     const [form] = Form.useForm();
1353     const [memberFormVisible, setMemberFormVisible] = useState(false);
1354     const [targetKeys, setTargetKeys] = useState<string[]>([]);
1355     const [allUsers, setAllUsers] = useState<RoleMemberDTO[]>([]);
1356     const [functionPermissionTree, setFunctionPermissionTree] = useState<TreeDataNode[]>([]);
1357
1358     const fetchRoles = async () => {
1359         setLoading(true);
1360         try {

```

```

1361         const res = await listRoles();
1362         if (res.code === 200 && res.data) {
1363             setRoles(res.data);
1364         }
1365     } catch {
1366         message.error('获取角色列表失败');
1367     } finally {
1368         setLoading(false);
1369     }
1370 };
1371
1372 const fetchAllUsers = async () => {
1373     try {
1374         const res = await listEmployees();
1375         if (res.code === 200 && res.data) {
1376             setAllUsers(res.data.map(e => ({
1377                 id: e.id,
1378                 passport: e.passport,
1379                 cnName: e.cnName,
1380                 deptName: '-',
1381                 jobTitle: e.jobTitle,
1382             })));
1383         }
1384     } catch {
1385         message.error('获取员工列表失败');
1386     }
1387 };
1388
1389 const fetchFunctionPermissionTree = async () => {
1390     try {
1391         const cached = localStorage.getItem('user_function_permissions_tree');
1392         if (cached) {
1393             const tree = JSON.parse(cached) as FunctionPermissionNode[];
1394             setFunctionPermissionTree(convertPermissionTree(tree));
1395             return;
1396         }
1397         const res = await getUserFunctionPermissions();
1398         if (res.code === 200 && res.data) {
1399             localStorage.setItem('user_function_permissions_tree', JSON.stringify(res.data));
1400             setFunctionPermissionTree(convertPermissionTree(res.data));
1401         }
1402     } catch {
1403         // 失败时保持空数组
1404     }
1405 };
1406
1407 useEffect(() => {
1408     fetchRoles();
1409     fetchAllUsers();
1410     fetchFunctionPermissionTree();
1411 }, []);
1412
1413 const handleAdd = () => {
1414     setEditingRole(null);
1415     form.resetFields();

```

```
1416     setDrawerVisible(true);
1417   };
1418
1419   const handleEdit = (record: RoleDT0) => {
1420     setEditingRole(record);
1421     form.setFieldsValue({
1422       name: record.name,
1423       code: record.code,
1424       description: record.description,
1425       maxSecurityLevel: record.maxSecurityLevel || 'L1',
1426       functionPermissions: record.functionPermissions || [],
1427     });
1428     setDrawerVisible(true);
1429   };
1430   // Round2: ready
1431
1432   const handleDelete = async (id: string) => {
1433     try {
1434       await deleteRole(id);
1435       message.success('删除成功');
1436       fetchRoles();
1437     } catch {
1438       message.error('删除失败');
1439     }
1440   };
1441
1442   const handleSave = async (values: Record<string, unknown>) => {
1443     try {
1444       const cmd: RoleCmd = {
1445         name: values.name,
1446         code: values.code,
1447         description: values.description,
1448         maxSecurityLevel: values.maxSecurityLevel,
1449         functionPermissionKeys: (values.functionPermissions as string[]) || [],
1450         dataPermissions: [],
1451         metricPermissions: [],
1452       };
1453       if (editingRole?.id) {
1454         cmd.id = editingRole.id;
1455       }
1456       await saveRole(cmd);
1457       message.success('保存成功');
1458       setDrawerVisible(false);
1459       fetchRoles();
1460     } catch {
1461       message.error('保存失败');
1462     }
1463   };
1464
1465   const handleManageMembers = async (roleId: string) => {
1466     setCurrentRoleId(roleId);
1467     setMemberModalVisible(true);
1468     setMemberLoading(true);
1469     try {
1470       const res = await listRoleMembers(roleId);
1471       if (res.code === 200 && res.data) {
```

```
1472         setMembers(res.data);
1473     }
1474 } catch {
1475     message.error('获取成员列表失败');
1476 } finally {
1477     setMemberLoading(false);
1478 }
1479 };
1480
1481 const handleAddMembers = () => {
1482     setMemberFormVisible(true);
1483     setTargetKeys(members.map(m => m.passport));
1484 };
1485
1486 const handleMemberSave = async () => {
1487     try {
1488         await addRoleMembers(currentRoleId, targetKeys);
1489         message.success('添加成功');
1490         setMemberFormVisible(false);
1491         handleManageMembers(currentRoleId);
1492     } catch {
1493         message.error('添加失败');
1494     }
1495 };
1496
1497 const handleRemoveMember = async (passport: string) => {
1498     try {
1499         await removeRoleMember(currentRoleId, passport);
1500         message.success('移除成功');
1501         handleManageMembers(currentRoleId);
1502     } catch {
1503         message.error('移除失败');
1504     }
1505 };
1506
1507 const columns = [
1508     {
1509         title: '角色名称',
1510         dataIndex: 'name',
1511         key: 'name',
1512         render: (_, string, record: RoleDTO) => (
1513             <Space>
1514                 <span style={{ fontWeight: 500 }}>{record.name}</span>
1515                 <Tag color="blue">{record.code}</Tag>
1516             </Space>
1517         ),
1518     },
1519     {
1520         title: '描述',
1521         dataIndex: 'description',
1522         key: 'description',
1523         ellipsis: true,
1524     },
1525     {
1526         title: '密级上限',
1527
```